STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC-26-0538	Darth Vader

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (**SMPS**)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code.** The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

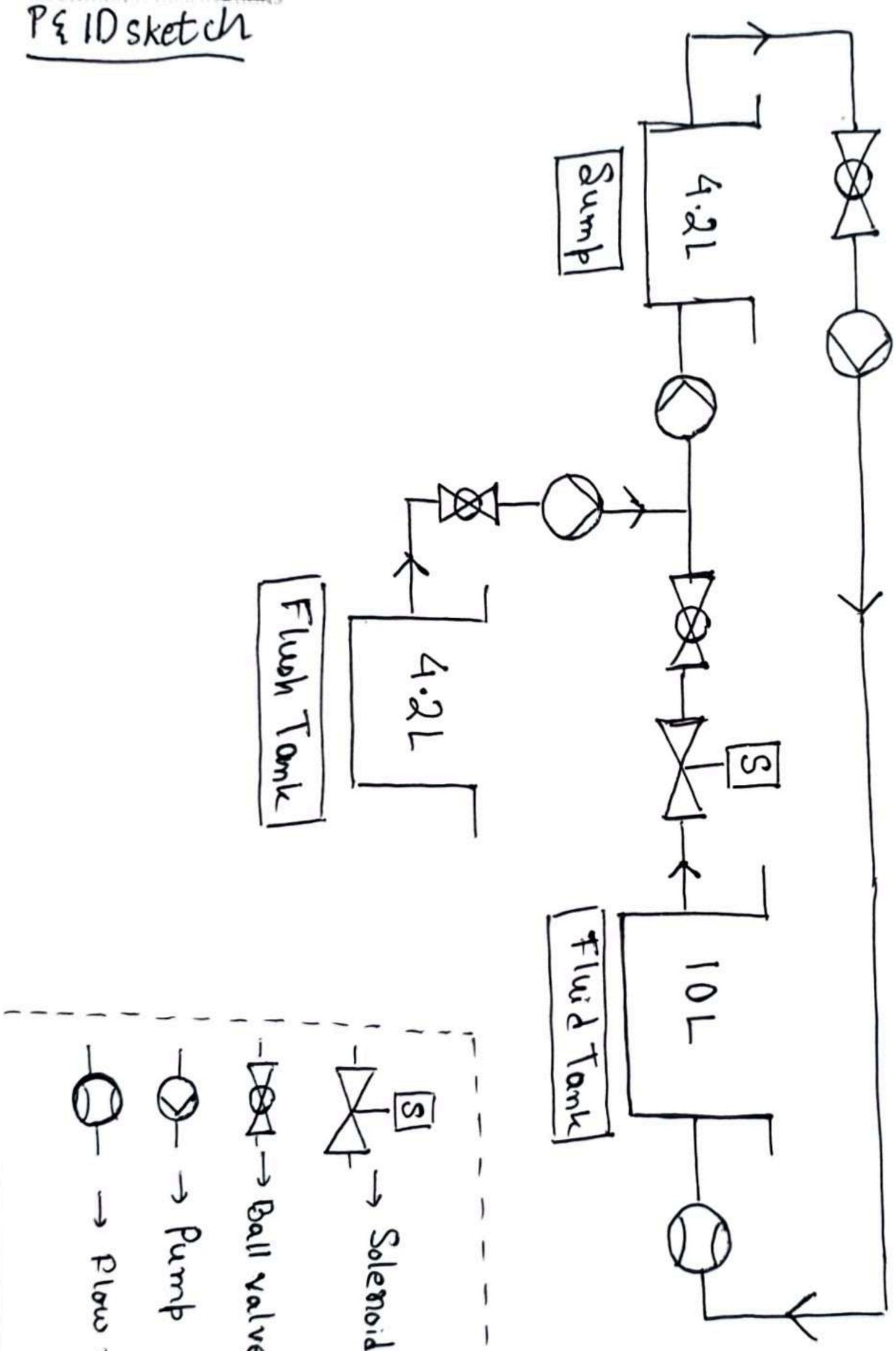
SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

~~CALCULATIONS~~

P&ID sketch



Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

The pumps provided cannot reach 6 L/min. That along with the fact that the flow meter works better at lower speeds led us to using the measured rates of ≈ 2.2 L/min, ~~1.08 L/min~~ 1.078 L/min, 1.53 L/min.

$A = 5.02655 \times 10^{-5}\ m^2$

~~Theoretical velocities : V_{th1} =~~

Flow rate Q (L/min)	Flow rate (m^3/s)	Theoretical vel. (m/s)	Actual Vel. (m/s)
~~1.078~~	1.7967×10^{-5}	0.357	0.351
1.53	2.55×10^{-5}	0.507	0.481
2.2	3.67×10^{-5}	0.73	0.666

It ~~er~~ was observed that actual flow velocity was less than theoretical velocity. This happens due to frictional losses, measurement errors, instrument error etc.

Flow Rate and Accumulated Volume Graphs

Team ID: LRC-26-0538

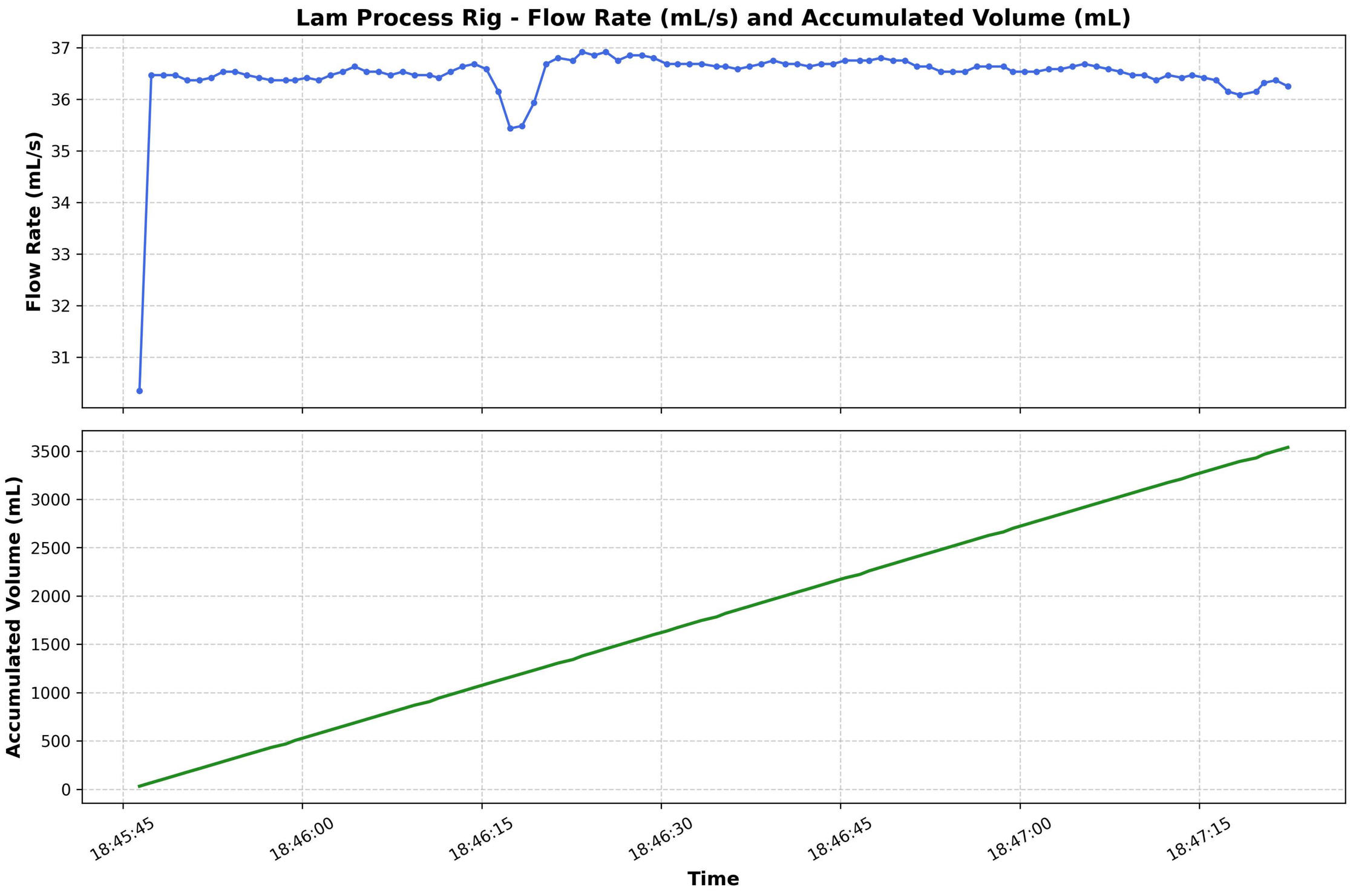


Figure 1: Lam Process Rig - Flow Rate (mL/s) and Accumulated Volume (mL)

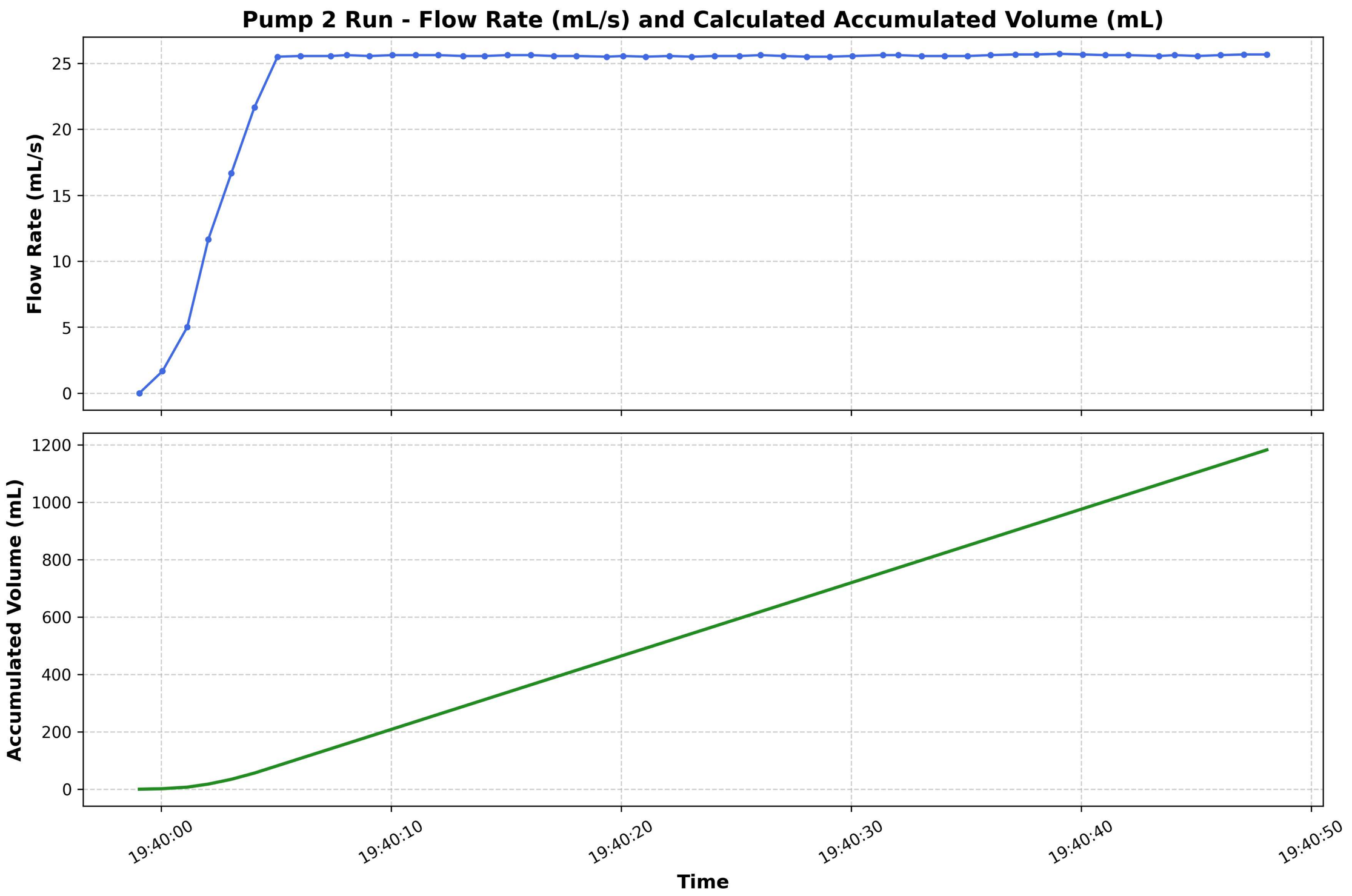
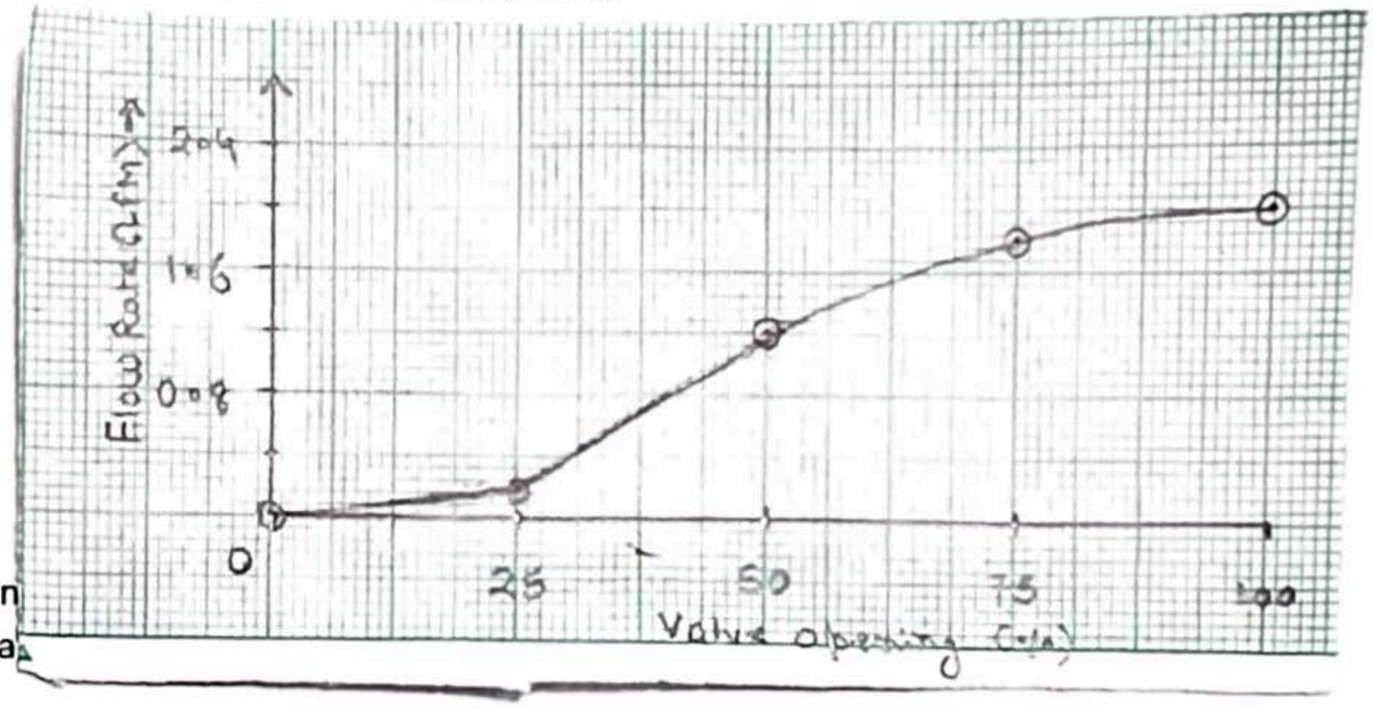


Figure 2: Pump 2 Run - Flow Rate (mL/s) and Calculated Accumulated Volume (mL)

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.2 L/min
50%	1.2 L/min
75%	2.0 L/min
100%	2.2 L/min



Q2.2 Determine the experimental calibration
Hz, ***a*** = experimental calibration coefficient a

Report:

R^2 = 0.99545

a = 0.0098259

b = 0.0135073

Maximum calibration error = 7.25%

Average calibration error = 2.50%

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	5940	27	220	1	2.222
2	6171	33	187	1	1.818
3	6272	49	128	1	1.224
4	6161	61	101	1	0.984
5	3862	60	64.7	0.7	0.700

HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

Answer is on next Page.

Given :- Effective liquid volume V = 0.5L

Flow rate, Q = 2.5L/min

Hydraulic residence time (T_r) = $\frac{V}{Q} = \frac{0.5}{2.5} = 0.2$ min

$\Rightarrow T_r = 12$ seconds

1) Residence time represents the average time the liquid spends inside the tubing before flowing out assuming steady flow and ideal mixing conditions.

2) If the volume doubles to 1.0L

$T_r = \frac{1.0}{2.5} * 60 = 24$ seconds (it doubles)

The residence time is directly proportional to the liquid volume

3) If flow rate doubles to 5.0L/min.

$T_r = \frac{0.5}{5.0} \times 60 = 6$ seconds (it halves)

the residence time is inversely proportional to the flow rate

4) Residence time determines how quickly a change in flow or fluid conditions at the inlet is reflected at the outlet. A longer residence time generally causes a greater delay in the measured response and vice-versa. This helps to distinguish actual changes in the system from delays caused by the tubing.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

Condition A → Normal operation

Condition B → Valve restriction

Condition C → Possible pump failure

Condition D → Pump Switched off

Condition E → Unstable flow → unstable pump operation
→ air bubbles, intermittent sensor connection

Most difficult condition to distinguish using FT-01 is Condition C, because zero flow may result from a hydraulic problem or a faulty flow sensor/signal

→ Additional information required: check pump current and operating status, measure inlet and operating status, measure inlet and outlet ~~press~~ pressure inspect the tubing for blockage and verify the FT-01 power supply and output signal. An independent flow measurement can confirm whether fluid is actually flowing.

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces**.
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller**. It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

Arduino / ESP32 Code (1/5)

Team ID: LRC-26-0538

```
#include "BluetoothSerial.h"

// Check if Bluetooth is properly enabled
#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please enable Bluetooth in your ESP32 board settings.
#endif

BluetoothSerial SerialBT;

// --- Pump 1 Pin Definitions ---
const int L_EN1 = 13;
const int R_EN1 = 26;
const int L_PWM1 = 32;
const int R_PWM1 = 33;

// --- Pump 2 Pin Definitions ---
const int L_EN2 = 16;
const int R_EN2 = 17;
const int L_PWM2 = 18;
const int R_PWM2 = 19;

// --- Pump 3 Pin Definitions ---
const int L_EN3 = 21;
const int R_EN3 = 22;
const int L_PWM3 = 23;
const int R_PWM3 = 25;

// --- Solenoid & Sensors ---
const int SOLENOID_PIN = 27;
const int FLOAT_SWITCH = 35;
const int FLOW_SENSOR = 14;

// --- ESP32 PWM Configuration ---
const int pwmFreq = 5000;
const int pwmResolution = 8;

// --- State Variables ---
bool p1_active = false;
bool p2_active = false;
bool p3_active = false;
bool sol_active = false;

// --- Automation States ---
bool case1_active = false;
bool case2_active = false;
unsigned long case2StartTime = 0;
const unsigned long CASE2_DURATION = 65000; // 1 min 5 secs in milliseconds

// Float drop delay tracking for Case 1
bool waitingForLevelDropDelay = false;
unsigned long levelDropTimer = 0;
const unsigned long LEVEL_DROP_DELAY_TIME = 20000; // 20 seconds delay after float opens

// --- Speed Control Variables (2:1 Ratio Maintained) ---
int speedPercent = 100;

// --- Flow Sensor, Rolling Average & Volume Accumulation ---
volatile unsigned long pulseCount = 0;
bool continuousFlowStreaming = false; // Toggle state for 'f'
bool showFlowRateInLMin = false;      // Toggle state for 'l' (Hz vs L/min)

const int NUM_SAMPLES = 12;
float frequencySamples[NUM_SAMPLES];
int sampleIndex = 0;
unsigned long lastSampleTime = 0;
const unsigned long sampleInterval = 250; // Sample every 250ms

unsigned long lastPrintTime = 0;
const unsigned long printInterval = 1000; // Output once every second

// Volume integration variables
float accumulatedVolumeLiters = 0.0;

// String buffer for parsing multi-character commands
String inputString = "";
bool stringComplete = false;

// Interrupt Service Routine for Flow Sensor (Triggering on FALLING edge)
```

Arduino / ESP32 Code (continued, 2/5)

Team ID: LRC-26-0538

```
void IRAM_ATTR flowInterrupt() {
  pulseCount++;
}

// Helper function to configure pumps using explicit LEDC channels
void setupPump(int len, int ren, int lpwm, int rpwm, int chL, int chR) {
  pinMode(len, OUTPUT);
  pinMode(ren, OUTPUT);
  digitalWrite(len, HIGH);
  digitalWrite(ren, HIGH);

  ledcAttachChannel(lpwm, pwmFreq, pwmResolution, chL);
  ledcAttachChannel(rpwm, pwmFreq, pwmResolution, chR);

  ledcWrite(lpwm, 0);
  ledcWrite(rpwm, 0);
}

// Helper to update physical hardware with scaled 2:1 speeds
void updateHardware() {
  int p2_target_pwm = (255 * speedPercent) / 100;
  int p1_target_pwm = (127 * speedPercent) / 100;

  int p1_speed = p1_active ? p1_target_pwm : 0;
  int p2_speed = p2_active ? p2_target_pwm : 0;
  int p3_speed = p3_active ? 255 : 0;

  ledcWrite(L_PWM1, 0);
  ledcWrite(R_PWM1, p1_speed);

  ledcWrite(L_PWM2, p2_speed);
  ledcWrite(R_PWM2, 0);

  ledcWrite(L_PWM3, p3_speed);
  ledcWrite(R_PWM3, 0);

  digitalWrite(SOLENOID_PIN, sol_active ? HIGH : LOW);
}

void setup() {
  Serial.begin(115200);

  SerialBT.begin("Lam_Process_Rig");
  Serial.println("Bluetooth Device Started as 'Lam_Process_Rig'.");

  setupPump(L_EN1, R_EN1, L_PWM1, R_PWM1, 0, 1);
  setupPump(L_EN2, R_EN2, L_PWM2, R_PWM2, 2, 3);
  setupPump(L_EN3, R_EN3, L_PWM3, R_PWM3, 4, 5);

  pinMode(SOLENOID_PIN, OUTPUT);
  digitalWrite(SOLENOID_PIN, LOW);

  pinMode(FLOAT_SWITCH, INPUT_PULLUP);
  pinMode(FLOW_SENSOR, INPUT);

  attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR), flowInterrupt, FALLING);

  Serial.println("System Ready.");
  SerialBT.println("System Ready via Bluetooth.");
}

// Process single or multi-character commands
void processCommand(String cmd, Stream &out) {
  cmd.trim();
  if (cmd.length() == 0) return;

  if (cmd == "c1") {
    case1_active = !case1_active;
    case2_active = false;
    if (case1_active) {
      accumulatedVolumeLiters = 0.0; // Reset volume on new run
      sol_active = true;
      p1_active = true;
      p2_active = true;
      p3_active = false;
      updateHardware();
      Serial.println("case 1 started");
      SerialBT.println("case 1 started");
```

Arduino / ESP32 Code (continued, 3/5)

Team ID: LRC-26-0538

```
    } else {
      Serial.println("case 1 stopped");
      SerialBT.println("case 1 stopped");
    }
  }
  else if (cmd.length() == 1) {
    char input = cmd.charAt(0);
    switch (input) {
      case '1':
        p1_active = !p1_active;
        out.print("Pump 1 toggled: "); out.println(p1_active ? "ON" : "OFF");
        updateHardware();
        break;
      case '2':
        p2_active = !p2_active;
        out.print("Pump 2 toggled: "); out.println(p2_active ? "ON" : "OFF");
        updateHardware();
        break;
      case '3':
        p3_active = !p3_active;
        out.print("Pump 3 toggled: "); out.println(p3_active ? "ON" : "OFF");
        updateHardware();
        break;
      case 's':
        sol_active = !sol_active;
        out.print("Solenoid toggled: "); out.println(sol_active ? "OPEN" : "CLOSED");
        updateHardware();
        break;
      case '+':
        if (speedPercent <= 90) speedPercent += 10;
        out.print("Speed Increased. Level: "); out.println(speedPercent);
        updateHardware();
        break;
      case '-':
        if (speedPercent >= 20) speedPercent -= 10;
        out.print("Speed Decreased. Level: "); out.println(speedPercent);
        updateHardware();
        break;
      case 'f':
        continuousFlowStreaming = !continuousFlowStreaming;
        out.print("Continuous Flow Streaming Toggled: ");
        out.println(continuousFlowStreaming ? "ON" : "OFF");
        break;
      case 'l':
      case 'L':
        showFlowRateInLMin = !showFlowRateInLMin;
        out.print("Flow Display Unit: ");
        out.println(showFlowRateInLMin ? "L/min & mL/s (Using Q = 0.009826f + 0.01351)" : "Frequency (Hz)");
        break;
      default:
        break;
    }
  }
}

void handleIncomingStream(Stream &stream) {
  while (stream.available() > 0) {
    char inChar = (char)stream.read();
    if (inChar == '\n' || inChar == '\r') {
      stringComplete = true;
      processCommand(inputString, stream);
      inputString = "";
    } else {
      inputString += inChar;
      if (inputString.length() == 1 && (inputString == "1" || inputString == "2" || inputString == "3" ||
          inputString == "s" || inputString == "+" || inputString == "-" || inputString == "f" || inputString == "l"))
              {
        processCommand(inputString, stream);
        inputString = "";
      }
    }
  }
}

void loop() {
  // 1. Check Inputs (Serial & Bluetooth)
  if (Serial.available() > 0) handleIncomingStream(Serial);
  if (SerialBT.available() > 0) handleIncomingStream(SerialBT);
```

Arduino / ESP32 Code (continued, 4/5)

Team ID: LRC-26-0538

```
  // 2. Float Switch & Case 1 Automation Logic
  int currentFloatState = digitalRead(FLOAT_SWITCH);

  if (case1_active) {
    if (currentFloatState == LOW) {
      // Float switch triggered (sump full) -> Stop Pump 1, keep Pump 2 running
      if (p1_active) {
        p1_active = false;
        updateHardware();
        Serial.println("float switch triggered, pump1 stopped.");
        SerialBT.println("float switch triggered, pump1 stopped.");
      }
      waitingForLevelDropDelay = false;
    }
    else {
  // Float switch is open (level normal)
  if (!p1_active && !waitingForLevelDropDelay) {
    waitingForLevelDropDelay = true;
    levelDropTimer = millis();
  }

  // When the 20s post-drain buffer completes, transition automatically to c2
  if (waitingForLevelDropDelay && (millis() - levelDropTimer >= LEVEL_DROP_DELAY_TIME)) {
    waitingForLevelDropDelay = false;

    case1_active = false;
    case2_active = true;
    sol_active = false; // close solenoid for c2
    p1_active = true;
    p2_active = true;
    p3_active = true;
    updateHardware();
    case2StartTime = millis();

    Serial.println("case 2 started");
    SerialBT.println("case 2 started");
  }
    }
  }

  // 3. Case 2 Automation Logic
  if (case2_active) {
    if (millis() - case2StartTime >= CASE2_DURATION) {
      case2_active = false;
      p1_active = false;
      p2_active = false;
      p3_active = false;
      sol_active = false;
      updateHardware();
      Serial.println("case 2 finished. shutting off all pumps");
      SerialBT.println("case 2 finished. shutting off all pumps");
    }
  }

  // 4. Background Sampling & Volume Integration (Every 250ms)
  unsigned long currentMillis = millis();
  if (currentMillis - lastSampleTime >= sampleInterval) {
    unsigned long elapsedInterval = currentMillis - lastSampleTime;
    lastSampleTime = currentMillis;

    noInterrupts();
    unsigned long counts = pulseCount;
    pulseCount = 0;
    interrupts();

    float instantFreq = counts * (1000.0 / sampleInterval);

    frequencySamples[sampleIndex] = instantFreq;
    sampleIndex = (sampleIndex + 1) % NUM_SAMPLES;

    if (p1_active || p2_active || p3_active) {
      float flowLMin = (0.009826 * instantFreq) + 0.01351;
      if (flowLMin < 0) flowLMin = 0;
      float intervalVolume = flowLMin * ((float)elapsedInterval / 60000.0);
      accumulatedVolumeLiters += intervalVolume;
    }
  }
```

Arduino / ESP32 Code (continued, 5/5)

Team ID: LRC-26-0538

```
  // 5. Print Continuous Flow Output Every Second If Toggled ON
  if (continuousFlowStreaming && (currentMillis - lastPrintTime >= printInterval)) {
    lastPrintTime = millis();

    float sum = 0;
    for (int i = 0; i < NUM_SAMPLES; i++) {
      sum += frequencySamples[i];
    }
    float smoothedFreq = sum / NUM_SAMPLES;

    if (showFlowRateInLMin) {
      float flowLMin = (0.009826 * smoothedFreq) + 0.01351;
      if (flowLMin < 0) flowLMin = 0;
      float flowMLSec = flowLMin * 16.6667; // Conversion from L/min to mL/s

      Serial.print("Flow : ");
      Serial.print(flowLMin, 3); Serial.print(" L/min - ");
      Serial.print(flowMLSec, 2); Serial.print(" mL/s - ");
      Serial.print(accumulatedVolumeLiters, 4); Serial.println(" L ,\n");

      SerialBT.print("Flow : ");
      SerialBT.print(flowLMin, 3); SerialBT.print(" L/min - ");
      SerialBT.print(flowMLSec, 2); SerialBT.print(" mL/s - ");
      SerialBT.print(accumulatedVolumeLiters, 4); SerialBT.println(" L ,\n");
    } else {
      Serial.print("Flow (Hz) : ");
      Serial.print(smoothedFreq, 1); Serial.print(" Hz - ");
      Serial.print(accumulatedVolumeLiters, 4); Serial.println(" L ,\n");

      SerialBT.print("Flow (Hz) : ");
      SerialBT.print(smoothedFreq, 1); SerialBT.print(" Hz - ");
      SerialBT.print(accumulatedVolumeLiters, 4); SerialBT.println(" L ,\n");
    }
  }
}
```